Cluster:NTS

TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING,

Pulchowk Campus

A PROJECT PROPOSAL ON

"Kathmandu Bus Route Finder"

A Web-Based Public Transport Navigation System for Kathmandu Valley

Submitted by

Dinesh Bhatta (PUL080BCT025)

Dipesh S Saud (PUL080BCT026)

Janak S Pujara (PUL080BCT035)

A

BE MINOR PROJECT PROPOSAL

SUBMITTED TO THE

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

LALITPUR, NEPAL

JUNE, 2026

Abstract

The Kathmandu Valley is served by an extensive but informally organized network of buses, microbuses, and tempos that carry the majority of daily commuters, yet no single, reliable digital system currently exists to help a commuter plan a journey across this network. This project, the Kathmandu Bus Route Finder, presents a full-stack web application that allows a user to specify an origin and a destination and receive a clear, navigable route across the Valley's bus network, including routes that require switching from one bus to another.

The system is built around three core components: a structured PostgreSQL/PostGIS database of bus stops and routes; a graph-based search engine implementing Dijkstra's algorithm as the primary pathfinding method for both direct and single-transfer routes, with BFS retained as an optional lightweight comparator for direct-route queries, using NetworkX; and an interactive Leaflet.js map interface that renders computed routes as road-following polylines via the OSRM routing engine. The backend is implemented in Python 3.11 using FastAPI, while the frontend is built with Next.js 14 and TypeScript.

The project directly addresses a consistent gap in existing tools: every current option for Kathmandu Valley commuters is either a route-search platform with insufficient local data, a static route map with no search capability, or a tool scoped to a single operator's fleet. No current system combines structured, queryable multi-operator route data with automatic transfer-aware origin-to-destination search. The Kathmandu Bus Route Finder targets a task success rate of at least 80% in informal user testing, an API response time under one second for direct-route queries and under two seconds for single-transfer queries and verified coverage of approximately 50 of the 85 identified major Kathmandu Valley route corridors (approximately 59%) at initial release, where a "major route" is defined as a DOTM-registered corridor operating at least hourly and passing through at least one of the nine principal interchange nodes of the Valley.

Table of Contents

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
BFS	Breadth-First Search
CI	Continuous Integration
CSS	Cascading Style Sheets
DB	Database
DOTM	Department of Transport Management
E2E	End-to-End
GTFS	General Transit Feed Specification
GiST	Generalized Search Tree
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IOE	Institute of Engineering
JSON	JavaScript Object Notation
JWT	JSON Web Token
NCSP	National Cyber Security Policy
OSM	OpenStreetMap
OSRM	Open Source Routing Machine
OWASP	Open Web Application Security Project
PostGIS	Post Geographic Information Systems (PostgreSQL spatial extension)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
TLS	Transport Layer Security
TU	Tribhuvan University
UAT	User Acceptance Testing

Abbreviation	Full Form
UI	User Interface
URL	Uniform Resource Locator
VAPT	Vulnerability Assessment and Penetration Testing

1. Introduction

The Kathmandu Valley is served by a dense but informally organized network of buses, microbuses, and tempos that together carry the large majority of daily commuters, yet no single, reliable digital system currently exists to help a commuter plan a journey across this network. Most residents rely on word-of-mouth knowledge built up over years of daily travel, while newcomers, students, and tourists are left guessing at bus numbers, boarding points, and transfer stops.

This project, the Kathmandu Bus Route Finder, is a full-stack web application that lets a user enter an origin and a destination and receive a clear, navigable route across the Valley's bus network, including routes that require switching from one bus to another. The system is built around three working parts: a structured database of stops and routes, a graph-based search engine that computes the shortest and most sensible path between two points, and an interactive map interface that displays the result visually rather than as a list of stop names. Together, these parts aim to turn local, informally held route knowledge into something any commuter can query in seconds.

This document expands on the original project proposal by setting out the problem in more detail, separating the project's objectives into a main goal and supporting specific goals, and reviewing the theoretical foundations and existing systems this project builds on or departs from, before identifying the specific gap the Kathmandu Bus Route Finder is intended to fill.

2. Problem Statements

Despite how heavily Kathmandu Valley commuters depend on public buses, the system that carries them remains almost entirely undocumented in any form a typical smartphone user could query. This project is motivated by the following specific, observable problems.

No authoritative digital route database: The Department of Transport Management and local transport committees hold route information, but it exists in fragmented, paper-based, or informal form rather than as a structured, publicly queryable dataset.

Difficulty identifying the correct bus: A commuter unfamiliar with a particular corridor has no reliable way to determine which bus number to board, where to board it, or where to get off, short of asking other passengers or conductors directly.

Incomplete coverage in general-purpose map tools: General-purpose mapping tools such as Google Maps support transit directions in cities with official GTFS feeds; in Kathmandu, where no such feed exists, their public-transport coverage is sparse and frequently outdated.

No support for multi-leg (transfer) journeys: When a single bus does not connect two points directly, working out a sensible transfer point and second route is left entirely to the commuter's own trial and error, with no tool to suggest a transfer route automatically.

Frequent, undocumented route changes: Bus routes, stop locations, and operating buses change periodically as operators adjust service, but no central, maintained source of truth exists to keep any digital record of the network current.

Collectively, these problems result in wasted commuter time, unnecessary reliance on comparatively expensive ride-hailing services for journeys a bus could serve, and a public transport system that is harder to use than its actual coverage would justify. A locally focused, graph-powered route finder, built on a deliberately collected and structured dataset, directly addresses this gap.

3. Objectives

3.1. Main Objective

The main objective of this project is to design and develop a web-based bus route finder for the Kathmandu Valley that allows a user to enter an origin and a destination and receive an accurate, visualised route, including transfer routes where no direct bus connects the two points, using a graph-based search over a structured, geospatially indexed dataset of local bus stops and routes.

3.2. Specific Objectives

(i) To design and build a full-stack web application in which a user can search for a bus route by specifying a source and a destination.

(ii) To model the Kathmandu Valley bus network as a weighted graph, with stops represented as nodes and route segments as edges, suitable for efficient pathfinding.

(iii) To implement Dijkstra's algorithm as the primary pathfinding method for both direct and single-transfer route queries, using a transfer-penalty weight at interchange nodes so that both route types are resolved in a single graph traversal. BFS is retained as an optional lightweight comparator for direct-route queries only.

(iv) To design and populate a PostgreSQL database with PostGIS spatial extensions, enabling efficient nearest-stop and proximity queries.

(v) To integrate Leaflet.js with OpenStreetMap base layers and the OSRM routing engine to render computed routes as road-following paths on an interactive map.

(vi) To collect, validate, and structure a working dataset covering 50 major Kathmandu Valley bus route corridors — organised across three priority tiers based on interchange-node connectivity — sufficient to demonstrate the system end-to-end and covering approximately 59% of the approximately 85 identified major corridors.

(vii) To evaluate the resulting system against defined accuracy, performance, and usability targets, as set out in Section 5.8 of this proposal.

(viii) To implement the system in compliance with Nepal's National Cyber Security Policy 2080 and the Electronic Transactions Act 2063, including enforcement of HTTPS/TLS encryption, JWT-based role-based access control on administrative endpoints, exclusion of user search-query data from logs and persistent storage, and a VAPT review of all public-facing API endpoints before deployment.

4. Literature Review

This section situates the Kathmandu Bus Route Finder within the broader body of theory and prior work on route-finding and public transit information systems, before identifying specifically where existing systems fall short for the Kathmandu Valley context.

4.1. Review of Related Theory

4.1.1 Graph Theory and Shortest-Path Search

A public transit network maps naturally onto a graph data structure: each stop is a node, and each direct segment of a route connecting two consecutive stops is an edge,

optionally weighted by distance, travel time, or fare. Two classical algorithms underpin most route-finding systems built on this model.

Breadth-First Search (BFS): explores a graph level by level outward from a source node and finds a route with the fewest stops when all edges are treated as equal weight [1]. However, fewest stops is not the same as most useful route in a real bus network — two paths between the same pair of stops may differ in stop count but be comparable in travel time. For this reason, BFS is retained in this project only as an optional lightweight comparator, while Dijkstra's algorithm applied to the full weighted graph is used as the primary search method for both direct and transfer queries.

Dijkstra's algorithm: computes the shortest weighted path from a source node to all other reachable nodes, and is the standard approach once edges carry meaningful weights such as distance or estimated travel time [2]. By assigning a transfer-penalty weight to interchange edges — connections between different routes at the same stop — a single Dijkstra traversal naturally surfaces both direct routes (zero-penalty paths) and optimal single-transfer routes simultaneously, without requiring two separate algorithm runs.

The single-transfer limit adopted in this project is a deliberate and acknowledged scope constraint. The Kathmandu Valley bus network is structured around nine principal interchange nodes — Ratnapark, Kalanki, Koteshwor, Lagankhel, Maharajgunj, New Baneshwor, Chabahil, Balaju, and Gongabu — through which the large majority of intra-Valley journeys can be completed with at most one transfer. Journeys requiring two or more transfers typically connect peripheral, low-frequency corridors and represent a minority of daily commuter trips. This limitation is disclosed to users when no result is returned within the single-transfer scope.

In practice, large-scale transit routers often extend these algorithms with transit-specific techniques such as the Connection Scan Algorithm (CSA) or RAPTOR, which exploit the timetabled, repeating structure of transit schedules to search faster than generic graph algorithms [3]. For a city-scale, frequency-based bus network without fixed timetables, however, Dijkstra's algorithm over a weighted stop-and-route graph remains an appropriate and well-understood foundation for this project's scale.

4.1.2 Geospatial Data Management

Storing and querying real-world coordinates efficiently requires more than a generic relational database can offer out of the box. Spatial database extensions, most notably PostGIS for PostgreSQL, add native geometry and geography types along with spatial indexing using R-tree based GiST indexes, allowing operations such as "find the nearest stop to this coordinate" or "find all stops within 500 metres" to run efficiently even as the dataset grows [4]. This theoretical grounding directly informs the choice of PostgreSQL with PostGIS as the project's data layer, discussed further in Section 5.3.

4.1.3 Web-Based Map Visualisation and Road-Network Routing

Beyond computing a sequence of stops, a usable route finder must present that route visually on a real map. Open-source web mapping libraries such as Leaflet.js render interactive maps from tile layers (commonly OpenStreetMap) and allow arbitrary routes, markers, and overlays to be drawn on top [5]. Translating a sequence of stop coordinates into a realistic, road-following path is itself a routing problem distinct from the transit-graph search described above, and is typically delegated to a dedicated road-network routing engine such as OSRM (Open Source Routing Machine), which computes the actual driving or walking path between consecutive stops along the real street network [6].

4.2. Review of Related Works

4.2.1 General-Purpose Transit Routing Platforms

Google Maps Transit is the most widely used journey-planning tool worldwide and supports multi-modal, multi-transfer routing in cities where transit agencies publish structured GTFS data. Its routing quality is directly dependent on the completeness of that underlying feed; in cities without an official, machine-readable transit feed, including Kathmandu, its public-transport directions are correspondingly sparse or unavailable [7]. Wikiroutes is a community-maintained, crowdsourced transit-map platform that includes a hand-curated map of Kathmandu Valley routes, stops, and fares, demonstrating both that demand for this information exists and that community mapping efforts can assemble it, though its primary interface is a static route map

rather than an origin-destination journey planner. More broadly, comparable South Asian cities facing similar transit data gaps have produced instructive cases. Dhaka's transit route digitization effort [9] and Colombo's metropolitan bus route standardization work [10] both demonstrate that structured, even partial, data collection enables meaningful journey planning tools — reinforcing that the key constraint for this project is data collection methodology, not algorithmic complexity.

4.2.2 Local Mapping and Transit Initiatives

The Yatayat project, originating from the 2012 Monsoon Collective initiative and continued with contributors from Kathmandu Living Labs, was an early effort to crowdsource Kathmandu Valley bus route and stop data directly into OpenStreetMap and to build a companion lookup web and Android application, after finding that official route data from the Department of Transport Management existed only in informal, paper-based form [8]. This project demonstrated both the feasibility and the difficulty of community-sourced transit data collection in the Valley, and its OpenStreetMap contributions remain a useful reference layer for any later mapping effort.

More recently, Sajha Yatayat, one of the Valley's larger cooperative bus operators, has released the Sajha Plus mobile application, which helps passengers identify which Sajha buses serve a given route. Coverage is limited to Sajha's own fleet rather than the wider Valley network. Several smaller commercial directory-style apps (such as Kathmandu Bus Route) provide static, hand-compiled lists of known routes and stops, but none offer automatic origin-to-destination search with transfer suggestions.

4.3. Gap Analysis

Table 4.3.1 below compares the Kathmandu Bus Route Finder against the systems reviewed above across the capabilities most relevant to a commuter trying to plan a journey.

Table 4.3.1: Capability comparison of existing route-finding tools

Capability	Google Maps Transit	Wikiroutes	Sajha Plus	Yatayat (OSM)	This project
Origin-destination route search	Yes (where GTFS exists)	No (static map only)	Partial (Sajha routes	No (lookup, not	Yes

Capability	Google Maps Transit	Wikiroutes	Sajha Plus	Yatayat (OSM)	This project
			only)	search)	
Kathmandu Valley route coverage	Sparse / incomplete	Broad, community-curated	Sajha fleet only	Broad (historic), unmaintained	50 defined corridors (~59% of ~85 major routes) at launch
Automatic transfer suggestion	Yes (where data exists)	No	No	No	Yes (single-transfer)
Interactive route map overlay	Yes	Yes (static)	Limited	Yes (historic)	Yes (Leaflet + OSRM)
Open, structured underlying data	No (proprietary)	Partially open	No (proprietary)	Yes (OSM)	Yes (PostgreSQL/PostGIS)
Actively maintained for Kathmandu	No	Community-dependent	Yes (operator-run)	No (largely inactive)	Yes (project scope)

This comparison points to a consistent gap: every existing option for Kathmandu Valley commuters is either a route-search tool with insufficient local data, a route map with no search function, or a search tool scoped to a single operator's fleet. No current system combines structured, queryable route data covering multiple operators with an automatic origin-to-destination search that can suggest a transfer when no direct bus exists. The Kathmandu Bus Route Finder is positioned directly in this gap.

5. Proposed Methodology

5.1. Project Approach

The Kathmandu Bus Route Finder is approached as an incremental, web-based geospatial application that solves a concrete civic problem: helping commuters in the Kathmandu Valley identify the most efficient public bus route between two points, including routes that require a transfer between buses. The overall approach is guided by three principles.

Problem-first, incremental delivery: The system is built outward from a minimal working core — a single-route search between two stops — before layering in transfer-based search, nearest-stop detection, and full map visualisation. This keeps the team able to demonstrate a working product at every milestone.

Graph-centric modelling: Bus routes and stops naturally form a graph (stops as nodes, route segments as edges), so the approach treats route-finding primarily as a graph search problem, separated cleanly from the web/API layer that exposes it.

Data realism: Because public transit data for the Kathmandu Valley is incomplete and fragmented, the approach commits to a structured, tiered data collection plan running in parallel with development across Weeks 1–5, with defined corridor targets and person-hour estimates rather than treating data as an afterthought.

At a high level, the project is organised into four layers: a data layer (route and stop data in PostgreSQL/PostGIS), a logic layer (NetworkX-based route graph and search algorithms), a service layer (FastAPI REST endpoints), and a presentation layer (Next.js + Leaflet.js map interface). This layered approach allows each part of the system to be developed, tested, and revised independently.

5.2. System Architecture & Design Modules

The system follows a three-tier architecture consisting of a frontend client, a backend application server, and a relational/geospatial database, with an external routing service used for road-network geometry. Figure 5.2.1 illustrates how these tiers interact.

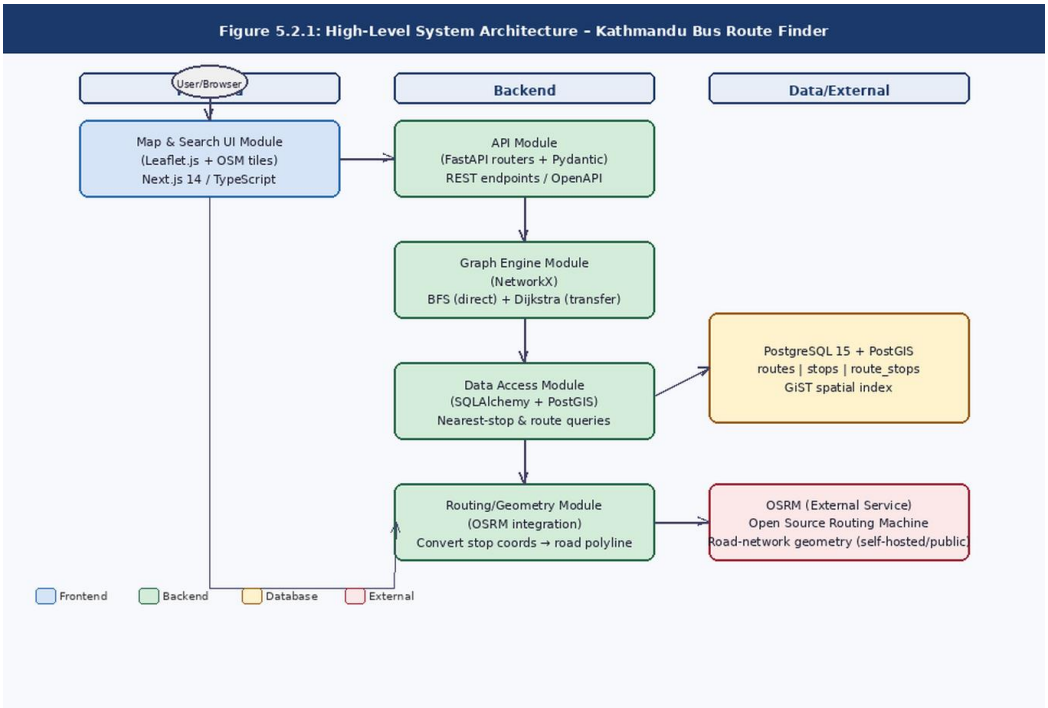

Figure 5.2.1: High-level system architecture of the Kathmandu Bus Route Finder

5.2.1 Design Modules

Map and Search UI Module: Renders the interactive map, search form (origin/destination), and the resulting route(s) as overlays using Leaflet.js with OpenStreetMap tiles.

API Module (FastAPI routers): Exposes REST endpoints for stop search, route search, and nearest-stop lookup; validates all requests and responses using Pydantic schemas and auto-generates OpenAPI documentation.

Graph Engine Module: Builds and holds an in-memory graph of stops and routes using NetworkX, and implements a unified Dijkstra-based search with transfer-penalty weights at interchange nodes, resolving both direct and single-transfer route queries in a single graph traversal.

Data Access Module: Manages persistent storage of stops, routes, and route-stop sequences, including spatial queries (e.g. nearest stop to a coordinate) via PostGIS geometry functions.

Routing/Geometry Module: A thin integration layer that calls the external OSRM service to convert a sequence of stop coordinates into a road-following polyline for display on the map.

5.3. Technology Stack

The technology stack was chosen to favour mature, well-documented, open-source tools that are free for academic use, that map cleanly onto the system's three-tier architecture, and that are compatible with the security requirements of Nepal's National Cyber Security Policy 2080 and the Electronic Transactions Act 2063. Table 5.3.1 summarises the choices by layer.

Table 5.3.1: Technology stack by layer

Layer	Technology	Purpose
Frontend	Next.js 14 (React, TypeScript)	Web application framework and UI rendering
Frontend	Leaflet.js	Interactive map rendering and route overlay
Frontend	Tailwind CSS	Styling and responsive layout
Backend	Python 3.11, FastAPI	REST API framework and request handling
Backend	NetworkX	Graph construction and Dijkstra shortest-path search
Backend	Pydantic	Request/response data validation
Database	PostgreSQL 15 + PostGIS	Relational storage and spatial queries
External service	OSRM (Open Source Routing Machine)	Road-network route geometry
Tooling	Git / GitHub	Version control and team collaboration
Tooling	Docker	Containerised local development and deployment
Tooling	Pytest, Jest	Backend and frontend automated testing

This stack was selected over alternatives (e.g. Express.js, Django, or a NoSQL store) primarily because PostGIS provides native, well-tested spatial query support that a document-based database would require building manually, and because FastAPI's

integration with Pydantic and automatic OpenAPI documentation accelerates iterative API development for a small team. All chosen technologies are open-source, well-maintained, and compatible with Nepal's National Cyber Security Policy 2080, which requires public-facing platforms to employ secure, regularly updated frameworks. HTTPS/TLS encryption will be enforced across all deployment environments; the chosen hosting providers (Vercel, Render) provision TLS certificates automatically, and correct enforcement will be explicitly verified and documented during the deployment phase. Environment variables and secrets are separated from source code and excluded from version control, in compliance with secure server configuration guidelines under the Electronic Transactions Act 2063. For production or official deployment beyond this academic project, hosting must be migrated to a Nepal-based or NTA/NTC-compliant data centre to satisfy in-country data sovereignty requirements.

5.4. Development Methodology

The project follows an iterative, Agile-inspired methodology rather than a strict Waterfall process, since requirements around data coverage and routing edge cases are expected to evolve as real bus route data is collected. Development is organised into short, fixed-length iterations of approximately one to two weeks each, with a working, demonstrable build at the end of every iteration. Figure 5.4.1 shows the overall project phase flow.

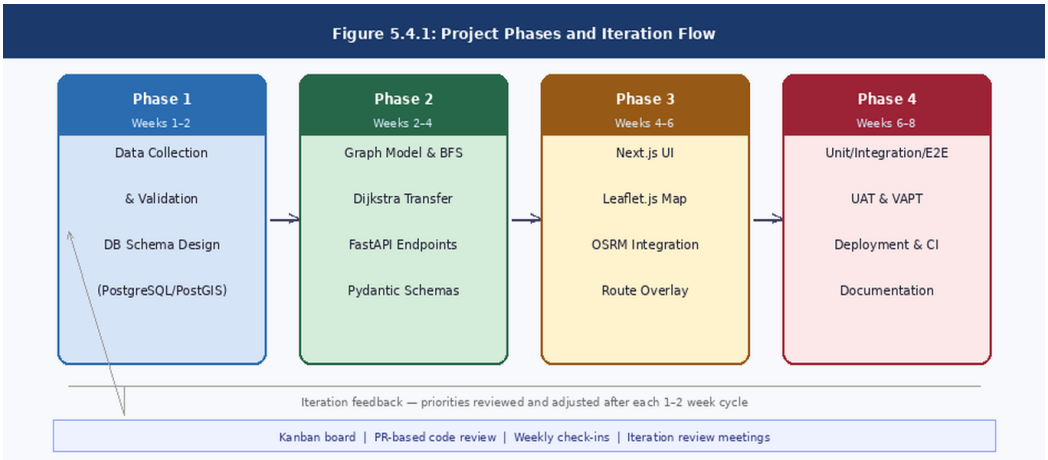

Figure 5.4.1: Project phases and iteration flow

5.4.1 Practices Adopted

Task boards: Tasks are tracked on a Kanban-style board (To Do / In Progress / Review / Done) to keep work visible across the team.

Peer code review: All code changes are submitted as pull requests and reviewed by at least one other team member before merging into the main branch.

Regular check-ins: Short check-ins at the start of each work session surface blockers early, in place of formal daily stand-ups given the team's size of three.

Iteration review: At the end of each iteration, the working build is reviewed against the original requirements and task priorities are adjusted for the next iteration accordingly.

5.5. Data Handling Strategy

Reliable route-finding depends entirely on the quality of the underlying stop and route data, so data handling is treated as a first-class concern rather than a preparatory step.

5.5.1 Route Scope and "Major Route" Definition

A "major route" is defined for this project as any bus corridor that: (a) is registered with the Department of Transport Management (DOTM); (b) operates at least hourly on weekdays; and (c) connects at least one of the nine principal interchange nodes of the Kathmandu Valley (Ratnapark, Kalanki, Koteshwor, Lagankhel, Maharajgunj, New Baneshwor, Chabahil, Balaju, Gongabu). A preliminary survey of OSM data and DOTM records identifies approximately 85 corridors meeting this definition. Feeder routes serving residential areas without passing through a principal interchange node are excluded from the initial release, though the database schema supports their addition.

Data collection targets 50 corridors at initial release, organised into three priority tiers:

- **Tier 1 (20 corridors):** Routes directly connecting two or more interchange nodes. Collected in Weeks 1–2. Estimated 40 person-hours.
- **Tier 2 (20 corridors):** Routes connecting one interchange node to a major neighbourhood (Patan, Bhaktapur, Budhanilkantha, Kirtipur). Collected in Weeks 2–4. Estimated 35 person-hours.
- **Tier 3 (10 corridors):** Remaining DOTM-registered corridors meeting the major-route definition. Collected in Weeks 4–5. Estimated 15 person-hours.

If Tier 3 collection is incomplete by Week 5, the minimum viable dataset of 40 Tier 1 + Tier 2 corridors is sufficient to demonstrate the system end-to-end. The coverage metric in Section 5.8 would be revised to $\geq 47\%$ in that scenario.

5.5.2 Data Sources and Collection

Primary data: Bus stop locations, route identifiers, and stop sequences for major routes within the Kathmandu Valley, collected manually and cross-checked against publicly available transit and mapping resources.

Secondary data: OpenStreetMap data is used as a reference for stop coordinates and road geometry where official sources are unavailable.

Structuring: Each route is recorded as an ordered sequence of stops with associated coordinates, stored in normalised form across three tables — routes, stops, and a route_stops join table — to avoid data duplication and to support efficient graph construction.

5.5.3 Data Quality and Validation

Coordinate values are validated to fall within the geographic bounds of the Kathmandu Valley before insertion. Duplicate stops within a small radius (e.g. 30 metres) are flagged for manual review and merged where appropriate. Each route is checked to ensure its stop sequence forms a connected, traversable path before being added to the live dataset.

5.5.4 Storage, Privacy, and Versioning

All route and stop data is stored in PostgreSQL with PostGIS spatial extensions, using indexed geometry columns to support efficient nearest-neighbour queries. The system does not collect or store personally identifiable user data as part of the core route-finding feature. In compliance with the data protection principles of the National Cyber Security Policy 2080, origin and destination search queries submitted by users are treated as sensitive location-pattern data: they are processed in memory to compute a route result and are not logged, retained in the database, or linked to any user identifier. No browser-side location history or search history is persisted by the frontend. A brief data handling notice will be displayed to users on first use, informing them that search queries are not stored. Schema changes are tracked through versioned migration scripts so that the dataset can be reproduced or rolled back consistently across development, testing, and deployment environments. Route data published on the platform will reflect only legally authorised routes as recognised by the Department of Transport Management; any route collected from informal or community sources will be cross-checked and flagged for review before inclusion.

5.6. Testing & Validation Plan

Testing follows a layered strategy, with the majority of test coverage concentrated at the unit level where logic is cheapest and fastest to verify, narrowing toward fewer, broader end-to-end checks. Figure 5.6.1 illustrates the testing pyramid, and Table 5.6.1 details the scope and example checks at each layer.

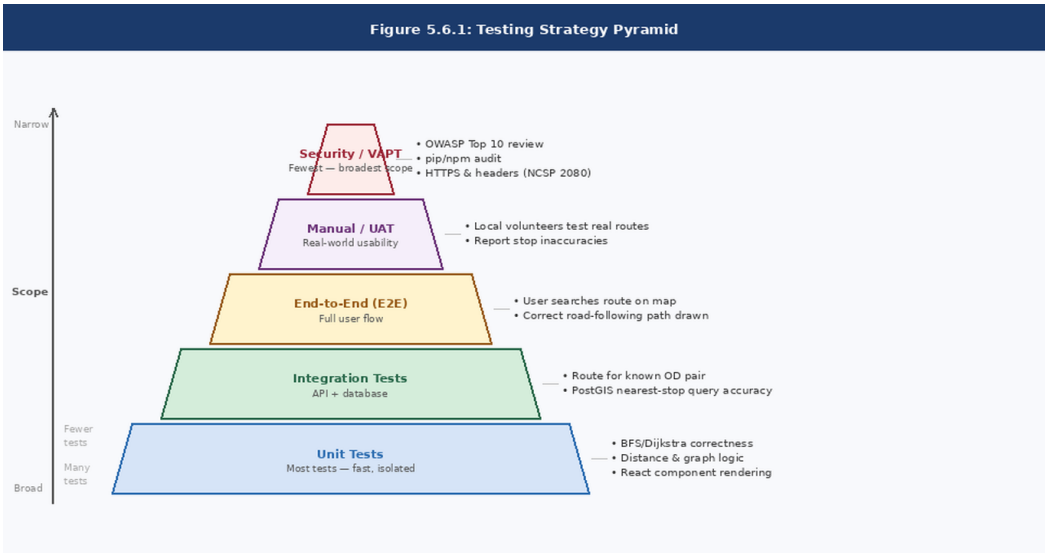

Figure 5.6.1: Testing strategy pyramid

Table 5.6.1: Testing strategy by layer and scope

Test type	Scope	Example checks
Unit tests	Individual functions / components	Dijkstra search correctness; transfer-penalty weight application; distance calculation; React component rendering
Integration tests	API endpoints + database	Correct route returned for a known origin/destination pair; PostGIS nearest-stop query accuracy
End-to-end (E2E) tests	Full user flow	User searches a route on the map UI and sees the correct path drawn on the map
Manual / UAT	Real-world usability	Minimum 15 participants drawn from at least three populations — Pulchowk Campus students unfamiliar with the tested routes, non-engineering staff or students, and at least 3 participants new to the Valley (tourists or recent arrivals). Participants complete a set journey task without prior training; success is observer-recorded.
Security / VAPT	API endpoints and dependencies	OWASP Top 10 review of all API endpoints; dependency vulnerability scan using pip audit and npm audit; manual check of HTTP headers (HTTPS enforcement, Content-Security-Policy, X-

Test type	Scope	Example checks
		Frame-Options). Mandated by Nepal's National Cyber Security Policy 2080 for public-facing web applications.

5.6.2 Validation Criteria

Route results are cross-checked against known, real-world bus routes for accuracy of stop sequence and transfer points. Only routes authorised by the Department of Transport Management (DOTM) are included; any route sourced from informal or community data is flagged for review before publication, in compliance with the Transport Management Directives. Search response time is measured to remain within acceptable thresholds (target: under 1 second for direct-route queries and under 2 seconds for single-transfer queries). Edge cases are explicitly tested: no route available, identical origin and destination, and stops with no nearby match within the dataset. Security validation criteria include: (a) all HTTP traffic redirected to HTTPS with a valid TLS certificate; (b) HTTP security headers present on all responses (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options); (c) no known high- or critical-severity vulnerabilities in Python or Node.js dependencies as reported by pip audit and npm audit; (d) no OWASP Top 10 issues identified in a manual review of all API endpoints; and (e) search query payloads confirmed absent from server logs and the database after a test session.

5.7. Deployment Plan

The application is planned to be containerised using Docker to ensure consistency between development and deployment environments, and deployed using free or low-cost tiers suitable for an academic project. Table 5.7.1 summarises the planned environment for each component. All deployment environments enforce HTTPS; Vercel and Render provision and renew TLS certificates automatically, and correct enforcement is verified as part of the security validation criteria defined in Section 5.6.2. Role-based access control (RBAC) is implemented on the backend administrative endpoints: only authenticated personnel with a valid JWT token issued at login may create, update, or delete route and stop data, satisfying the Zero-Trust access requirements of the National Cyber Security Policy 2080. It is acknowledged that the hosting providers selected for this academic project (Vercel, Render, Supabase) operate infrastructure outside Nepal. This is acceptable for a supervised

academic submission. Any transition to official public deployment would require migration to a Nepal-based or NTA/NTC-compliant data centre to fulfil in-country data sovereignty obligations under the National Cyber Security Policy 2080 and the Electronic Transactions Act 2063.

Table 5.7.1: Planned deployment environment by component

Component	Planned environment
Frontend (Next.js 14)	Vercel (free tier) or static export served via Nginx
Backend (FastAPI)	Containerised with Docker; deployed on Render or Railway free/hobby tier
Database (PostgreSQL + PostGIS)	Managed PostgreSQL instance (Render / Supabase) with PostGIS extension enabled
Version control / CI	GitHub repository with GitHub Actions for automated test runs on each pull request

Environment configuration (database URLs, API keys, JWT secrets) is managed through environment variables rather than hard-coded values, and kept out of version control using a .env file excluded from the repository. Administrative backend routes are protected with JWT-based authentication and role checks, so that only authorised users may modify route or stop data. A staging environment mirrors the production setup at a smaller scale to validate changes before release; security header checks and dependency audits are run in the staging environment as part of the deployment pipeline before any release is promoted to production. Deployments are rolled out in stages: backend and database first, followed by the frontend, to ensure the API is available before the client depends on it. These practices collectively satisfy the secure server configuration and role-based access requirements mandated by Nepal's National Cyber Security Policy 2080.

5.8. Evaluation Metrics

The success of the system will be evaluated against a combination of functional accuracy, performance, and usability metrics. These metrics will be revisited at the end of each development iteration, allowing the team to track progress quantitatively and adjust priorities accordingly. Table 5.8.1 summarises the planned targets.

Table 5.8.1: Evaluation metrics and targets

Category	Metric	Target
Functional accuracy	Correct route returned for known origin-destination pairs	≥ 90% of a blind test set of 20 origin-destination pairs assembled by the team member not involved in graph engine development
Functional accuracy	Correct identification of transfer points where no direct route exists	≥ 85% of test cases
Performance	• Average API response time for a direct-route search • Average API response time for a single-transfer search	• < 1 second under normal load • < 2 seconds under normal load
Performance	Map and route render time on the frontend	< 2 seconds on a typical mobile connection
Reliability	Backend test suite pass rate on each release build	100% passing
Usability	Task success rate in user testing (find a route end-to-end)	≥ 80% of a minimum 15 participants complete without assistance; participants drawn from non-expert populations including new Valley residents
Data coverage	Proportion of the ~85 identified major corridors in the dataset at launch	≥ 59% (~50 corridors); minimum viable release at 47% (~40 corridors) if Tier 3 collection is incomplete
Security	HTTPS enforced; TLS certificate valid; all HTTP traffic redirected to HTTPS	100% pass (binary)
Security	No high- or critical-severity vulnerabilities in pip audit / npm audit dependency scan	0 high/critical issues
Security	Search query payloads absent from server logs and database after a test session (privacy	100% pass (binary)

Category	Metric	Target
	compliance, NCSP 2080)	

6. Time Schedule

The project is planned over an eight-week period from the date of proposal approval. Table 6.1 below shows the Gantt-style distribution of major tasks across the eight weeks, with the designated lead member for each task. Shaded cells (▓) indicate active weeks for each task and blank dot (○) indicates a passive buffer slot, activated only if collection is behind.

Table 6.1: Eight-week project time schedule (Gantt)

Task	Lead	W1	W2	W3	W4	W5	W6	W7	W8
1a. Data Collection — Tier 1	Dinesh	▓	▓						
1b. Data Collection — Tier 2 & 3	All		▓	▓	▓	▓			
2. DB Schema Design (PostgreSQL/PostGIS)	Dipesh	▓	▓						
3. Graph Model & Dijkstra Implementation	Janak		▓	▓					
4. Dijkstra / Transfer Route Logic	Janak			▓	▓				
5. FastAPI Endpoints & Pydantic Schemas	Dipesh			▓	▓				
6. Next.js UI & Leaflet.js Map	Dinesh				▓	▓			
7. OSRM Integration & Route Overlay	Dinesh					▓	▓		
8. Unit & Integration Testing	Dipesh					▓	▓		
9. E2E Testing & UAT	All						▓	▓	

Task	Lead	W1	W2	W3	W4	W5	W6	W7	W8
10. Deployment & CI Setup	Janak							▓	▓
11. Documentation & Report Writing	All							▓	▓
12. VAPT & Security Review	All						▓	▓	
13. Contingency / Data Gap Recovery	All				○	○			

Data collection runs across Weeks 1–5 in a tiered structure, overlapping with algorithm and API development from Week 2 onwards. If collection is behind schedule at the end of Week 3, the contingency action is to reduce the release target to the 40 Tier 1 and Tier 2 corridors already collected and defer Tier 3; this minimum viable dataset is sufficient to demonstrate the system end-to-end. The graph engine and API are built in Weeks 2–4, followed by the frontend and map integration in Weeks 4–6. Testing, deployment, and documentation overlap in the final two weeks to ensure the system is validated and documented before the submission deadline. VAPT and security review (Task 12) runs concurrently in Weeks 7–8, covering the OWASP Top 10 endpoint review, pip audit and npm audit dependency scans, and verification of HTTPS enforcement and HTTP security headers, in compliance with the National Cyber Security Policy 2080.

7. Expected Outcomes

Upon successful completion, the Kathmandu Bus Route Finder is expected to deliver the following outcomes.

(i) **A working, publicly accessible web application** that allows any user to enter an origin and destination within the Kathmandu Valley and receive a direct or single-transfer bus route with an interactive map overlay showing the road-following path. When no route within the single-transfer scope exists, the system explicitly informs the user rather than returning a silent failure.

(ii) **A structured, open dataset** of major Kathmandu Valley bus stops and routes, stored in PostgreSQL/PostGIS format and covering approximately 50 corridors

(~59% of the approximately 85 identified major routes) at initial release, which can be independently extended or reused by subsequent projects.

(iii) A validated graph-based route engine implementing unified Dijkstra's algorithm with transfer-penalty weights for both direct and single-transfer route queries, with a documented API and an automated test suite achieving 100% pass rate on release builds.

(iv) Demonstrated usability with at least 80% of a minimum 15 non-expert user-test participants — drawn from at least three populations including new Valley residents — successfully completing an end-to-end journey query without assistance and an API response time under one second for direct-route queries.

(v) Academic deliverables including this project proposal document, a final project report covering design, implementation, testing, and evaluation, and a presentation to the Department of Electronics and Computer Engineering at IOE Pulchowk Campus.

(vi) A security-compliant deployment demonstrating adherence to Nepal's National Cyber Security Policy 2080 and the Electronic Transactions Act 2063: HTTPS/TLS enforced across all environments, JWT-based role-based access control on administrative endpoints, zero retention of user search-query data, and a completed VAPT review with no high- or critical-severity findings. More broadly, the project is intended to demonstrate that a structured, graph-powered digital tool for informal urban transit networks is feasible, practically useful, and deployable in a manner compliant with Nepal's cyber security and transport management regulatory framework, and to serve as a reference implementation that future work can extend with real-time data, additional transfer legs, fare information, or mobile-native interfaces.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[2] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, 1959.

[3] D. Delling, T. Pajor, and R. F. Werneck, "Round-based public transit routing," Transportation Science, vol. 49, no. 3, pp. 591–604, 2015.

[4] P. Ramsey and M. Obe, PostGIS in Action, 3rd ed. Shelter Island, NY: Manning Publications, 2021.

[5] Leaflet.js contributors, "Leaflet — an open-source JavaScript library for mobile-friendly interactive maps," 2024. [Online]. Available: https://leafletjs.com

[6] Project OSRM contributors, "Open Source Routing Machine: Modern C++ routing engine for shortest paths in road networks," 2024. [Online]. Available: http://project-osrm.org

[7] Google LLC, "Google Maps Platform — Transit directions," 2024. [Online]. Available: https://developers.google.com/maps/documentation/directions

[8] Kathmandu Living Labs, "Yatayat — Mapping Kathmandu's public transport network," 2013. [Online]. Available: http://kathmandulivinglabs.org/projects/yatayat

[9] Bangladesh Road Transport Authority (BRTA), "Dhaka transit route digitisation study," Dhaka, Bangladesh, 2022.

[10] Sri Lanka Transport Board, "Colombo metropolitan bus route standardisation report," Colombo, Sri Lanka, 2021.

[11] OpenStreetMap contributors, "OpenStreetMap," 2024. [Online]. Available: https://www.openstreetmap.org

[12] Sajha Yatayat, "Sajha Plus mobile application," 2023. [Online]. Available: https://sajhayatayat.com.np

[13] Government of Nepal, Ministry of Communication and Information Technology, "National Cyber Security Policy 2080 (2023)," National Cyber Security Centre, Kathmandu, Nepal, 2023. [Online]. Available: https://ncsc.gov.np/content/10889/national-cyber-security-policy-english/

[14] Government of Nepal, "Electronic Transactions Act, 2063 (2008)," Kathmandu, Nepal, 2008. [Online]. Available: https://radiantca.com.np/assets/nav_file/Electronic%20Transaction%20Act%202063.pdf

[15] Department of Transport Management (DOTM), Government of Nepal, "Transport Management Directives," Kathmandu, Nepal. [Online]. Available: https://dotm.gov.np/